



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: Victor David Perez Argueta **Carné:** 1074325

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. **¿Qué es una API REST?**

Es un medio que permite que diferentes aplicaciones se comuniquen entre sí mediante Internet, utilizando solicitudes y respuestas.

2. **¿Qué es un endpoint?**

Es una dirección específica dentro de una API que permite acceder a una función o recurso determinado.

3. **¿Qué diferencia existe entre una URL y un endpoint?**

Una URL es una dirección web en general, mientras que un endpoint es una URL específica que pertenece a una API y realiza una acción determinada.

4. **¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?**

GET sirve para consultar información, POST para crear datos, PUT o PATCH para modificar información y DELETE para eliminarla.

5. **¿Qué es JSON y por qué es común en las APIs?**

JSON es un formato utilizado para organizar y transmitir información. Es común porque es sencillo de leer y fácil de utilizar en diferentes lenguajes de programación.

6. **¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.**

Es un número que indica el resultado de una solicitud. El código 200 significa que fue exitosa, 404 indica que el recurso no fue encontrado y 500 representa un error interno del servidor.

7. **¿Qué información se puede encontrar en la respuesta de una API?**

Puede incluir los datos solicitados, un código de estado, encabezados y mensajes relacionados con el resultado de la solicitud.

8. **¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?**

Permite comprobar fácilmente si los endpoints funcionan correctamente, observar las respuestas e identificar errores antes de implementar la API en un programa.

9. **¿Qué es y para qué sirve Swagger UI?**

Es una herramienta que muestra de forma visual la documentación de una API. Permite conocer sus endpoints, métodos, parámetros y probar solicitudes.

10. **¿Qué función cumple Postman en el desarrollo y prueba de APIs?**

Postman permite enviar solicitudes a una API, revisar sus respuestas y verificar que las diferentes operaciones funcionen correctamente.



1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

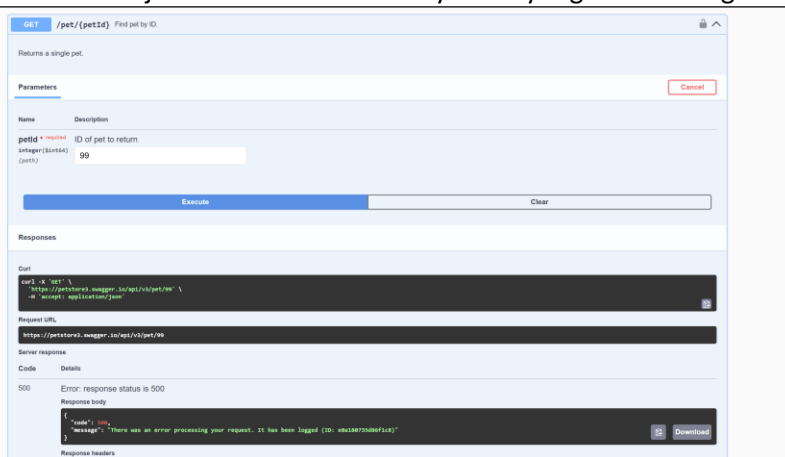
API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingrese a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
- Indique el método HTTP utilizado.
- Indique qué parámetro necesita la solicitud.
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.



- Explique con sus palabras qué información contiene la respuesta.

la solicitud no pudo completarse correctamente, ya que ocurrió un error al procesar la petición. El sistema devuelve el código 500 junto con un mensaje indicando que el error fue registrado.

Evidencia: captura de Swagger mostrando la operación y su respuesta.



3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id

name

status

category

tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.

Edit Value | Schema

```
{
  "id": 10743,
  "name": "doggie",
  "category": {
    "id": 1,
    "name": "vp"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'https://petstore3.swagger.io/api/v3/pet' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 10743,
    "name": "doggie",
    "category": {
      "id": 1,
      "name": "vp"
    },
    "photoUrls": [
      "string"
    ],
    "tags": [
      {
        "id": 0,
        "name": "string"
      }
    ],
    "status": "available"
  }'
```



Server response

Code	Details
200	Response body <pre>{ "id": 10743, "category": { "id": 1, "name": "vp" }, "name": "doggie", "photoUrls": ["string"], "tags": [{ "id": 0, "name": "string" }], "status": "available"}</pre> Response headers <pre>access-control-allow-headers: Content-Type,api_key,Authorization access-control-allow-methods: GET,POST,DELETE,PUT access-control-allow-origin: * access-control-expose-headers: Content-Disposition content-length: 138 content-type: application/json date: Fri,02 Oct 2026 22:59:29 GMT server: Jetty(9.4.53.v20231008)</pre>

Name	Description
petId * required integer(\$int64) (path)	ID of pet to return 10743

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/10743' \
  -H 'accept: application/json'
```

Request URL

```
https://petstore3.swagger.io/api/v3/pet/10743
```

Server response

Code	Details
200	Response body <pre>{ "id": 10743, "category": { "id": 1, "name": "vp" }, "name": "doggie", "photoUrls": ["string"], "tags": [{ "id": 0, "name": "string" }], "status": "available"}</pre> Response headers <pre>access-control-allow-headers: Content-Type,api_key,Authorization access-control-allow-methods: GET,POST,DELETE,PUT access-control-allow-origin: * access-control-expose-headers: Content-Disposition content-length: 138 content-type: application/json date: Fri,02 Oct 2026 23:01:58 GMT server: Jetty(9.4.53.v20231009)</pre>



- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué?
 - Se utilizó el método PUT, porque permite modificar la información de una mascota que ya existe.
- ¿Qué datos fueron enviados en el body?
 - Se enviaron los datos de la mascota en formato JSON. Se mantuvo el ID 10743 y se modificaron el nombre a **doggie2** y el estado a **pending**.
- ¿Qué código HTTP devolvió la API?
 - La API devolvió el código HTTP **200**, indicando que la solicitud fue procesada correctamente.
- ¿Cómo comprobó que el cambio realmente ocurrió?
 - Se comprobó realizando nuevamente una consulta con **GET /pet/{petId}** y utilizando el ID 10743. La respuesta mostró el nuevo nombre y el nuevo estado, confirmando que la modificación se realizó correctamente.

Responses

Curl

```
curl -X 'PUT' \
  'https://petstore3.swagger.io/api/v3/pet' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 10743,
    "category": {
      "id": 1,
      "name": "vp"
    },
    "name": "doggie2",
    "photoUrls": [
      "string"
    ],
    "tags": [
      {
        "id": 0,
        "name": "string"
      }
    ],
    "status": "pending"
  }'
```

Request URL

https://petstore3.swagger.io/api/v3/pet

Server response

Code	Details
200	Response body <pre>{ "id": 10743, "category": { "id": 1, "name": "vp" }, "name": "doggie2", "photoUrls": ["string"], "tags": [{ "id": 0, "name": "string" }], "status": "pending" }</pre> Response headers <pre>access-control-allow-headers: Content-Type,api_key,Authorization access-control-allow-methods: GET,POST,DELETE,PUT access-control-allow-origin: * access-control-expose-headers: Content-Disposition content-length: 137 content-type: application/json date: Fri,02 Oct 2026 23:04:44 GMT server: Jetty(9.4.53.v20231009)</pre>



5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.
 - El parámetro utilizado fue status.
- Explique dónde se envía ese parámetro.
 - El parámetro se envía en la dirección de la solicitud para indicar qué estado se desea consultar.
- Observe la estructura de la respuesta.
- Indique qué sucede si cambia el valor del parámetro por pending.
 - Al cambiar el parámetro a pending, la API devuelve las mascotas que tienen ese estado en lugar de las que están disponibles.

Consulta de mascotas con estado available.

status * required

string

(query)

Status values that need to be considered for filter

available

Execute

Clear

Responses

Curl

curl -X 'GET' \

'https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available' \

-H 'accept: application/json'

Request URL

https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available

Server response

Code

Details

200

Response body

[

{

"id": 12345,

"category": {

"id": 1,

"name": "dogs"

},

"name": "fluffy",

"photoUrls": [],

"tags": [

{

"id": 0,

"name": "tag1"

},

{

"id": 1,

"name": "tag2"

},

],

"status": "available"

},

{

"id": 123456,

"category": {

"id": 1,

"name": "Dogs"

},

"name": "Oleh",

},

]

Response headers

access-control-allow-headers: Content-Type,api_key,Authorization

access-control-allow-methods: GET,POST,DELETE,PUT

access-control-allow-origin: *

access-control-expose-headers: Content-Disposition

content-type: application/json

date: Fri,02 Oct 2026 23:09:49 GMT

server: Jetty(9.4.53.v20231009)

Responses

Consulta de mascotas con estado pending.



status * **required** Status values that need to be considered for filter

string

(query)

Execute **Clear**

Responses

Curl

```
curl -X 'GET' \
'https://petstore3.swagger.io/api/v3/pet/findByStatus?status=pending' \
-H 'accept: application/json'
```

Request URL

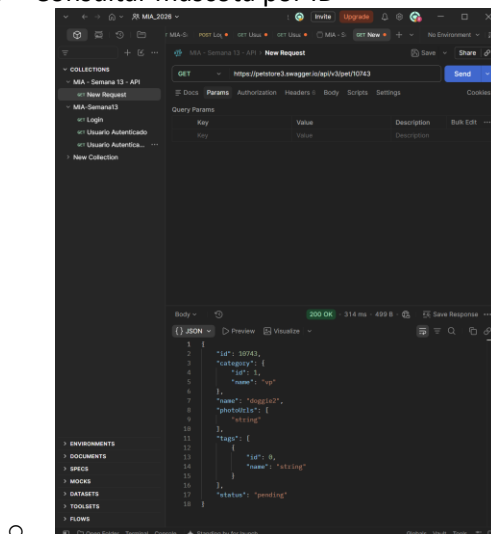
```
https://petstore3.swagger.io/api/v3/pet/findByStatus?status=pending
```

Server response

Code	Details
200	Response body <pre>[{ "id": 108, "category": { "id": 10800, "name": "Perros" }, "name": "Kobe", "photoUrls": ["string"], "tags": [{ "id": 108, "name": "Doki" }], "status": "pending" }, { "id": 1012, "category": { "id": 1, "name": "Dogs" }, "name": "doggie", "photoUrls": ["string"], "status": "pending" }]</pre> Response headers <pre>access-control-allow-headers: Content-Type,api_key,Authorization access-control-allow-methods: GET,POST,DELETE,PUT access-control-allow-origin: * access-control-expose-headers: Content-Disposition content-type: application/json date: Fri,02 Oct 2026 23:11:15 GMT server: Jetty(9.4.53.v20231009)</pre>

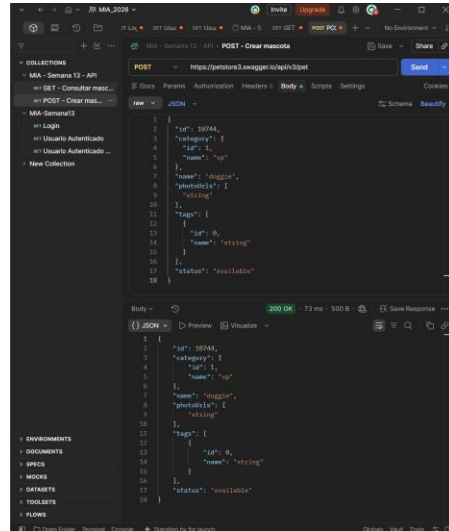
6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID

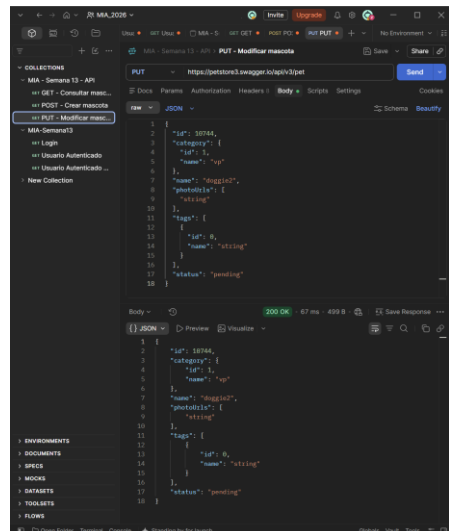




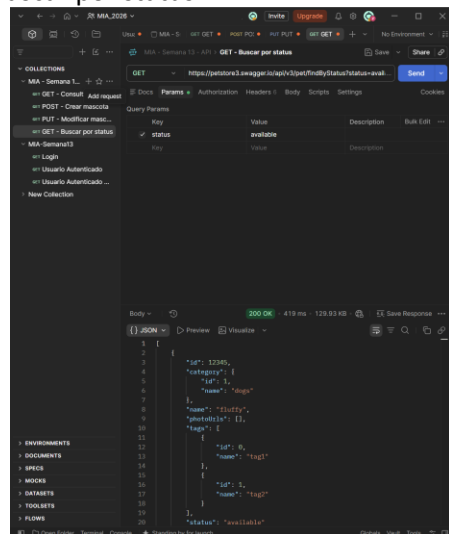
- POST – Crear mascota



- PUT – Modificar mascota

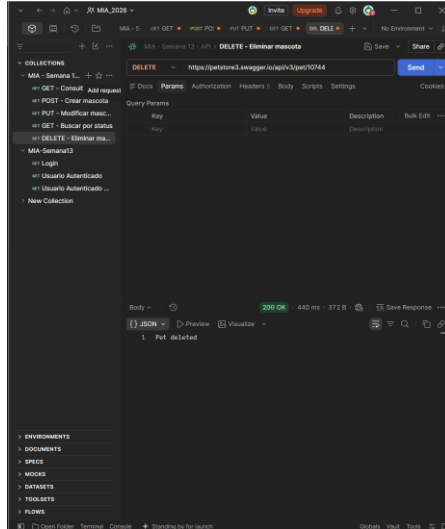


- GET – Buscar por status





- **DELETE – Eliminar mascota**



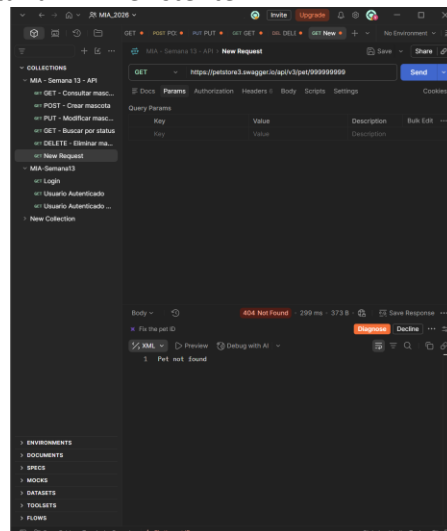
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.



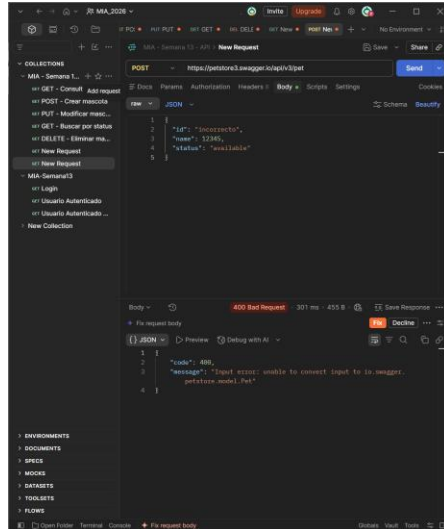
- a) **Solicitud realizada:** Se realizó una solicitud GET utilizando el ID 999999999.
- b) **Código HTTP recibido:** 404 Not Found.
- c) **Respuesta de la API:** Pet not found.
- d) **Qué esperaba que ocurriera:** Se esperaba que la API indicara que no existía una mascota con ese ID.



e) **Qué ocurrió realmente:** La API devolvió un error 404 indicando que la mascota no fue encontrada.

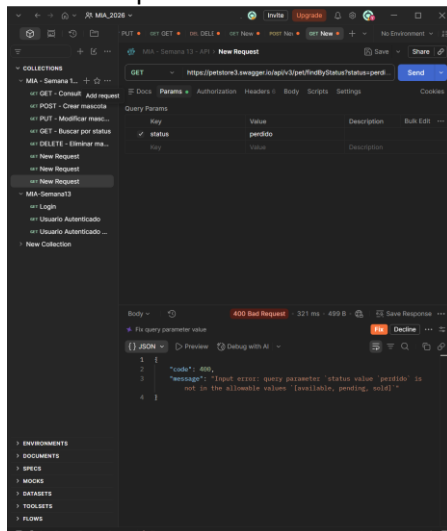
f) **Explicación de la causa:** Esto ocurrió porque el ID utilizado no corresponde a una mascota registrada.

2. Enviar un JSON incorrecto o incompleto.



- a) **Solicitud realizada:** Se realizó una solicitud POST enviando un JSON con datos incorrectos.
- b) **Código HTTP recibido:** 400 Bad Request.
- c) **Respuesta de la API:** La API indicó que no pudo procesar correctamente los datos enviados.
- d) **Qué esperaba que ocurriera:** Se esperaba que la API rechazara la solicitud debido a que los datos no tenían el formato correcto.
- e) **Qué ocurrió realmente:** La API rechazó la solicitud y devolvió un error 400.
- f) **Explicación de la causa:** El error ocurrió porque se enviaron valores con tipos incorrectos, por ejemplo un texto en el campo id, donde se esperaba un número.

3. Utilizar un parámetro con un valor no esperado.



- a) **Solicitud realizada:** Se realizó una solicitud GET utilizando el parámetro status con el valor perdido.
- b) **Código HTTP recibido:** 400 Bad Request.
- c) **Respuesta de la API:** La API indicó que el valor perdido no está dentro de los valores permitidos para el parámetro status.
- d) **Qué esperaba que ocurriera:** Se esperaba que la API rechazara la solicitud porque el valor ingresado no era válido.



e) **Qué ocurrió realmente:** La API rechazó la solicitud y devolvió un error 400.

f) **Explicación de la causa:** El error ocurrió porque el parámetro status solo acepta los valores available, pending y sold.

4. Intentar eliminar un recurso que ya no existe.

Para cada prueba registre:

- Solicitud realizada.
- Código HTTP recibido.
- Respuesta de la API.
- Qué esperaba que ocurriera.
- Qué ocurrió realmente.
- Explicación de la causa

8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

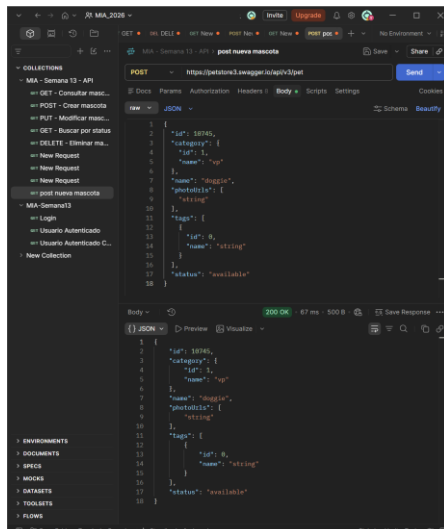
- Defina qué endpoint utilizaría para cada operación.

Operación	Método	Endpoint
Registrar mascota	POST	/pet
Consultar mascota	GET	/pet/{petId}
Actualizar mascota	PUT	/pet
Eliminar mascota	DELETE	/pet/{petId}
Comprobar existencia	GET	/pet/{petId}

- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
 - El programa debe analizar el código HTTP y el contenido de la respuesta. Esto permite identificar si la operación fue exitosa o si ocurrió algún problema, como intentar consultar una mascota que ya no existe.
- Ejecute el flujo completo en Postman.

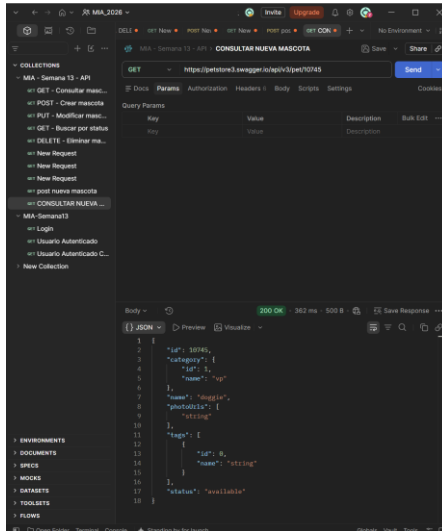
Capture evidencia de cada paso.

1. CREACION MASCOTA

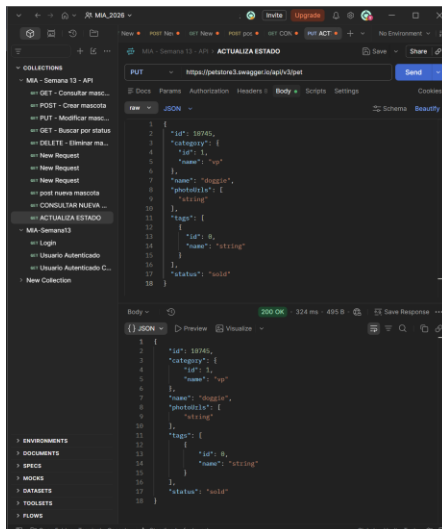




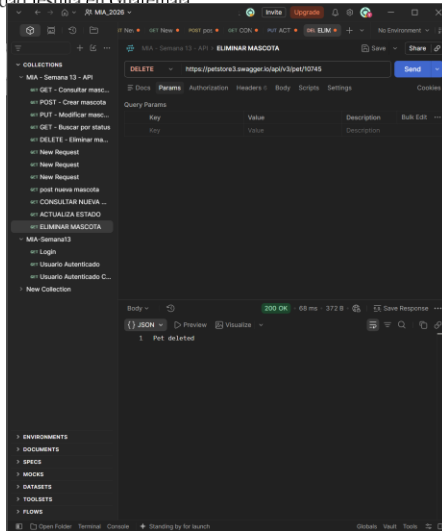
2. CONSULTAR MASCOTA



3. ACTUALIZAR ESTADO



4. ELIMINAR MASCOTA



Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.